Phase4 ConfusionBuffer Anki

Phase 4 — Confusion Buffer & Anki Pack (Full-Stack / Production AI Systems + MLOps)

Companion to the Phase-4 daily schedule (Weeks 37–42). Systems + design, not derivations — defend architectural choices and design the next-gen version. *(Expanded edition.)*

How to use: Block-A concept → cards + glossary → explain the trade-off aloud next morning. **Triangulation targets:** *KV-cache & why decode is memory-bound*, *how to evaluate a RAG system honestly*, *training-serving skew*, *what ZeRO shards*, *continuous batching*.

Ranked hard-topics map

1. **RAG done right** — retrieval quality, reranking, and honest evaluation.
2. **ML system design** — the four layers + drift + retraining loop.
3. **Inference optimization** — KV-cache, batching, quantization.
4. **Distributed training** — DP/TP/PP, ZeRO/FSDP, mixed precision.
5. **Evaluation & observability** — LLM-as-judge pitfalls, online monitoring.

Anki deck (`Q → A`)

Deck A · LLM internals

- **Q:** What is the KV-cache and why does it matter? → **A:** Caches past keys/values so each new token's attention reuses them → O(n) per token, not O(n²); its size grows with sequence length (a memory/bandwidth cost at inference).
- **Q:** Why are prefill and decode different regimes? → **A:** Prefill processes the whole prompt in parallel (compute-bound); decode emits one token at a time (memory-bandwidth-bound — KV-cache reads dominate).
- **Q:** BPE tokenization — idea + why it matters for cost? → **A:** Merge frequent byte/char pairs into subword tokens; token count drives context length, latency, and $/1k-tokens.
- **Q:** Sampling: greedy / temperature / top-k / nucleus? → **A:** Greedy=argmax (repetitive); temperature scales logits (higher=random); top-k keeps k highest; nucleus (top-p) keeps the smallest set with cumulative prob ≥ p.
- **Q:** Beam search vs sampling — when? → **A:** Beam (keep b best partial sequences) for closed-ended, high-precision tasks (translation); sampling for open-ended/creative generation.
- **Q:** Context window — the practical limits? → **A:** Max tokens the model attends to; longer = more cost (attention scales) and dilution/"lost in the middle"; not a free substitute for retrieval.

Deck B · RAG at senior level

- **Q:** Chunking trade-off? → **A:** Small chunks = precise retrieval but lost context; large = context but diluted relevance/embedding quality; overlap preserves boundaries.
- **Q:** Dense vs BM25 vs hybrid retrieval? → **A:** Dense embeddings capture semantics; BM25 captures exact terms/rare tokens; hybrid (fusion, e.g., RRF) gets both; rerank refines the top-k.
- **Q:** Vector index types (HNSW vs IVF)? → **A:** HNSW = graph-based ANN (fast, high-recall, more memory); IVF = inverted-file clustering (memory-efficient, tunable nprobe). Both approximate nearest-neighbor for scale.
- **Q:** What does a reranker add? → **A:** A cross-encoder scores (query, doc) jointly → higher precision than bi-encoder retrieval; applied to the retrieved candidates.
- **Q:** How do you evaluate RAG (RAGAS)? → **A:** Score retrieval (context precision/recall) and generation (faithfulness, answer-relevance) *separately*; faithfulness = is the answer grounded in retrieved context.
- **Q:** Contextual retrieval? → **A:** Prepend chunk-specific context (a short doc/section summary) before embedding each chunk → better retrieval of ambiguous chunks.
- **Q:** RAG vs fine-tuning — decision rule? → **A:** RAG for changing/large knowledge + citations; fine-tune for a consistent skill/format/style; combine when you need both knowledge and behavior.
- **Q:** Two sources of RAG hallucination + fixes? → **A:** Retrieval miss (better retriever/rerank/hybrid) and ungrounded generation (grounding prompt + a faithfulness critic + citations).
- **Q:** Where does a critic/verifier improve RAG? → **A:** A second pass checks the answer against sources (faithfulness), catches hallucinations, and can trigger re-retrieval — the evaluator-optimizer pattern (mirrors your critic-agent project).

Deck C · MLOps & ML system design

- **Q:** The four layers of an ML system? → **A:** Data → feature (engineering/store) → model (train/registry) → serving (inference/monitoring), with orchestration + observability across all.
- **Q:** Training-serving skew? → **A:** Features computed differently at train vs serve → silent accuracy drop; fix with a shared feature pipeline/store.
- **Q:** Data drift vs concept drift + detection? → **A:** Data drift = input distribution shifts (monitor PSI/KL on features); concept drift = input→output relationship shifts (monitor labeled performance) → both feed retraining triggers.
- **Q:** When to retrain? → **A:** On schedule, on drift detection, or on performance dropping below SLA — balancing freshness vs cost/stability.
- **Q:** What goes in a model registry + why version everything? → **A:** Versioned models + metadata/lineage/metrics; version data+code+config too for reproducibility and safe rollback.
- **Q:** Batch vs online vs streaming inference? → **A:** Batch = precomputed (throughput, stale); online = per-request (fresh, latency-sensitive); streaming = continuous updates.
- **Q:** Shadow vs canary deployment? → **A:** Shadow = run the new model on real traffic without serving its output; canary = serve to a small traffic slice; both de-risk rollout (with rollback).
- **Q:** What is data validation (e.g., Great Expectations) for? → **A:** Assert schema/range/null/distribution expectations on incoming data → catch pipeline breakage and silent data corruption early.

- **Q:** SLA vs SLO vs SLI? → **A:** SLI = a measured indicator (p99 latency); SLO = the target for it; SLA = the contractual promise (with consequences).

Deck D · Distributed training

- **Q:** Data vs tensor vs pipeline parallelism? → **A:** Data = replicate model, split the batch, all-reduce grads; tensor = split a layer's matmul across devices; pipeline = split layers into stages. Combine ("3D parallelism") for very large models.
- **Q:** What does ZeRO / FSDP shard? → **A:** Optimizer states, gradients, and parameters across data-parallel ranks → fit larger models without full replication.
- **Q:** Mixed precision (AMP / bf16) — why + the catch? → **A:** Compute in fp16/bf16 for speed+memory; keep an fp32 master copy / loss-scaling (fp16) to preserve numerics; bf16 has more range, less mantissa.
- **Q:** Gradient accumulation — what for? → **A:** Sum grads over several micro-batches before stepping → simulate a large batch on limited memory.
- **Q:** Activation checkpointing (recomputation)? → **A:** Discard activations on the forward pass and recompute them in backward → trade compute for memory (train bigger models/longer sequences).
- **Q:** Pipeline "bubble" — what is it? → **A:** Idle time as stages fill/drain the pipeline; micro-batching reduces the bubble fraction.

Deck E · Inference optimization

- **Q:** vLLM / paged attention? → **A:** Manages the KV-cache in non-contiguous "pages" (OS-virtual-memory style) → less fragmentation, higher batched throughput.
- **Q:** Static vs dynamic vs continuous batching? → **A:** Static = fixed batch; dynamic = wait-and-group by a window; continuous (in-flight) = add/remove requests each step (vLLM) → best throughput+latency for LLMs.
- **Q:** Quantization families (weight-only vs activation; GPTQ/AWQ/GGUF)? → **A:** Weight-only (GPTQ/AWQ) shrinks memory with small loss; activation quant (INT8) also speeds compute; GGUF = a CPU/edge-friendly format. Calibration reduces the loss.
- **Q:** Speculative decoding? → **A:** A small draft model proposes several tokens, the big model verifies them in one pass → fewer big-model steps → lower latency, same distribution.
- **Q:** Distillation vs pruning vs quantization? → **A:** Distillation = train a small student from a big teacher; pruning = remove weights/heads; quantization = lower precision — all shrink/accelerate, often combined.
- **Q:** Throughput vs latency? → **A:** Batching raises throughput but can raise per-request latency; continuous batching balances both; profile p50/p95/p99.

Deck F · Evaluation & observability

- **Q:** LLM-as-judge pitfalls + mitigations? → **A:** Position/verbosity/self-preference bias and inconsistency; mitigate with rubric prompts, pairwise comparison, randomized order, and human spot-checks.
- **Q:** Why do offline metrics ≠ production quality? → **A:** Distribution shift, prompt/model changes, and user behavior; you need online monitoring + feedback loops.
- **Q:** What is a golden/eval dataset + regression testing? → **A:** A curated set of representative inputs+expected behavior; re-run it on every prompt/model change to catch regressions before shipping.
- **Q:** What does tracing/observability capture for LLM apps? → **A:** Per-request spans (retrieval, prompt, tokens, latency, cost, tool calls) → debug, monitor drift, and audit — mind PII in logs.

Common misconceptions & traps

- **"RAG fixes hallucination."** Only if the answer is *faithful* to retrieved context and verified — otherwise it hallucinates with citations.
- **"Bigger model = better in production."** Latency, cost, and serving constraints often make a smaller/quantized model the right call.
- **"Offline eval = production quality."** Monitor online; distributions shift.
- **"Quantization is free."** It trades accuracy; calibrate and measure.
- **"A long context window removes the need for RAG."** Cost scales, attention dilutes ("lost in the middle") — retrieval still helps.
- **"Accuracy is the serving metric."** Latency/throughput/cost/drift are first-class; a p99 breach can matter more than a point of accuracy.
- **"Retrain more often = better."** Retraining adds cost, risk, and non-determinism; trigger on drift/SLA, not vibes.

Glossary starter

KV-cache · prefill/decode · BPE · sampling (greedy/top-k/nucleus/temperature) · beam search · context window / lost-in-the-middle · chunking/overlap · dense vs BM25 vs hybrid (RRF) · HNSW/IVF (ANN) · reranker (cross-encoder) · RAGAS / faithfulness / context precision-recall · contextual retrieval · RAG vs fine-tune · feature store · training-serving skew · data vs concept drift (PSI/KL) · model registry / lineage · shadow/canary/rollback · data validation · SLI/SLO/SLA · data/tensor/pipeline (3D) parallelism · ZeRO/FSDP · mixed precision (AMP/bf16, loss scaling) · gradient accumulation · activation checkpointing · pipeline bubble · vLLM/paged attention · static/dynamic/continuous batching · quantization (GPTQ/AWQ/GGUF) · speculative decoding · distillation/pruning · LLM-as-judge · golden dataset / regression testing · tracing/observability.

Drills

Whiteboard: design a full train→serve→monitor loop with the four layers + drift triggers + rollback; how you'd evaluate a RAG system honestly (retrieval vs generation); where a critic-agent adds value; data vs tensor vs pipeline parallelism; the KV-cache memory picture. **Blank-file:** a hybrid-retrieval + rerank RAG with a RAGAS harness on public docs; a KV-cache; a drift-detection check (PSI); quantize + benchmark a small model (p50/p95/p99); an LLM-as-judge eval with randomized order.

Leaving bar (cold, no notes)

The KV-cache + why decode is memory-bound; evaluate a RAG system (retrieval vs generation, faithfulness, RAG-vs-fine-tune); the four-layer ML system + training-serving skew + drift/retraining + rollback; DP/TP/PP + what ZeRO shards + mixed precision; continuous batching + a quantization family + speculative decoding; LLM-as-judge pitfalls + why online ≠ offline; and a 45-min ML-system-design for a molecular-property service with calibrated UQ.